

STUDENT MANAGEMENT SYSTEM

PROJECT REPORT

1. Title of the Project

Student Management System – A CRUD-Based Full-Stack Web Application

2. Abstract

The **Student Management System** is a web-based application developed to manage student information efficiently. The system allows users to **add, view, update, delete, and search student records** through a simple and user-friendly interface.

The application follows a full-stack architecture consisting of a **frontend, backend REST API, and database**. The frontend is developed using **HTML, CSS, and JavaScript**, while the backend is implemented using **Java Spring Boot**. **MySQL** is used as the database for storing student information.

The system implements the four fundamental CRUD operations: **Create, Read, Update, and Delete**. Input validation is included to reduce incorrect data entry, and REST API endpoints are used for communication between the frontend and backend.

The project demonstrates how modern web technologies can be integrated to develop a simple, functional, and maintainable student information management system.

3. Introduction

Managing student information manually can become difficult when the number of students increases. Maintaining records using notebooks or separate files can lead to duplication, data loss, and difficulties in searching or updating information.

The **Student Management System** provides a digital solution for storing and managing student records. Users can enter student details through a web interface and perform different operations on the stored records.

The project follows the CRUD-based web application development process specified in the given SOP, including frontend development, backend REST API development, database connectivity, validation, testing, and documentation.

4. Problem Statement

Traditional student record management methods have several limitations:

- Student information may be stored manually.
- Searching for a particular student can take time.
- Updating information manually is difficult.
- Records may be duplicated.
- Deleting outdated records can be inconvenient.
- There may be a higher possibility of human errors.

Therefore, a simple web-based system is required to manage student records efficiently.

5. Objectives

The main objectives of the Student Management System are:

1. To develop a simple web-based student management application.
2. To store student information in a database.
3. To implement **Create, Read, Update, and Delete** operations.
4. To provide a simple and user-friendly interface.
5. To provide search functionality.
6. To validate important student information.
7. To connect the frontend with a Spring Boot REST API.
8. To test the application using API testing tools.
9. To maintain the project using Git and GitHub.
10. To provide proper project documentation.

These objectives correspond to the SOP's requirement for a functional full-stack CRUD application with frontend, backend, REST API, database, validation, testing, and documentation.

6. Scope of the Project

The system can be used for basic student record management.

The project includes:

- Student registration
- Student record viewing
- Student record updating
- Student record deletion
- Student search
- Database storage
- REST API
- Input validation
- API testing
- Responsive web interface

The SOP identifies **Student Management System** as one of the suitable CRUD application examples.

7. Technologies Used

Technology	Purpose
HTML	Creating webpage structure
CSS	Designing the user interface
JavaScript	Frontend functionality
Java	Backend programming
Spring Boot	Developing REST API
Spring Data JPA	Database communication
MySQL	Storing student data
Postman	API testing
VS Code	Development environment
Git	Version control
GitHub	Source code repository

The SOP recommends HTML/CSS/JavaScript for frontend development, Spring Boot for backend development, MySQL/PostgreSQL/SQLite for databases, Postman for API testing, and Git/GitHub for version control.

8. System Requirements

Hardware Requirements

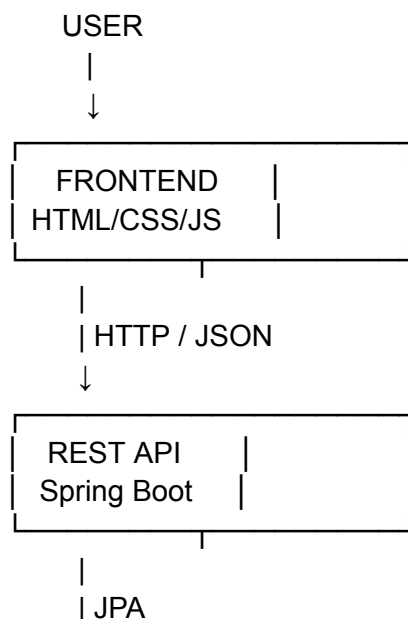
- Computer/Laptop
- Minimum 4 GB RAM
- Minimum 10 GB free storage
- Internet connection
- Keyboard and mouse

Software Requirements

- Windows/Linux/macOS
 - Visual Studio Code
 - Java JDK 17
 - Spring Boot
 - MySQL Server
 - MySQL Workbench
 - Postman
 - Web browser
 - Git
-

9. System Architecture

The system follows a simple three-layer architecture:





The SOP describes the same general flow of **User** → **Frontend** → **REST API** → **Spring Boot** → **ORM/JPA** → **Database**.

10. Modules of the System

The Student Management System contains the following major modules.

10.1 Student Registration

The user enters student information such as:

- Student Name
- Email
- Department
- Year
- Phone Number

The information is submitted to the backend and stored in MySQL.

10.2 Student Viewing

The system retrieves student records from the database and displays them in a table.

10.3 Student Updating

The user can select a student and modify existing information.

10.4 Student Deletion

The user can delete an existing student record.

10.5 Student Search

The search box allows users to search students using details such as:

- Name
- Email
- Department

11. Database Design

The project uses a **MySQL database** named:

student_management

The main table is:

students

Student Table

Field	Data Type	Description
id	BIGINT	Primary Key
name	VARCHAR	Student name
email	VARCHAR	Student email
department	VARCHAR	Department
year	INT	Current year
phone	VARCHAR	Phone number

Primary Key

id uniquely identifies every student.

Unique Constraint

The **email** field can be configured as unique so that duplicate email records are avoided.

The SOP requires database tables, attributes, primary keys, relationships, and constraints to be identified during database design.

12. CRUD Operations

CRUD represents four basic database operations.

12.1 Create

Used to add a new student.

POST /api/students

Example:

```
{  
  "name": "Thiru",  
  "email": "thiru@gmail.com",  
  "department": "Information Technology",  
  "year": 2,  
  "phone": "9876543210"  
}
```

12.2 Read

Used to retrieve student records.

Get all students

GET /api/students

Get a particular student

GET /api/students/{id}

12.3 Update

Used to modify an existing student record.

PUT /api/students/{id}

12.4 Delete

Used to remove a student record.

DELETE /api/students/{id}

The SOP specifies the standard REST mapping of POST, GET, PUT/PATCH, and DELETE for CRUD operations.

13. REST API

The backend provides REST API endpoints for communication between the frontend and database.

Method	Endpoint	Operation
POST	<code>/api/students</code>	Create student
GET	<code>/api/students</code>	Read all students
GET	<code>/api/students/{id}</code>	Read one student
PUT	<code>/api/students/{id}</code>	Update student
DELETE	<code>/api/students/{id}</code>	Delete student

The frontend communicates with these endpoints using JavaScript `fetch()` and JSON data. This follows the SOP's frontend-backend integration approach.

14. Frontend Design

The frontend contains three main files:

```
frontend
├── index.html
├── style.css
└── script.js
```

index.html

Provides:

- Student form
- Search box
- Student table
- Edit button
- Delete button

style.css

Provides:

- Layout
- Input styling
- Button styling
- Table styling
- Responsive design

script.js

Handles:

- API requests
- Adding students
- Displaying students
- Editing students
- Deleting students
- Searching students

The SOP requires the frontend to provide forms, record display, edit/delete functionality, validation, and search/filter functionality.

15. Backend Design

The backend is developed using **Java Spring Boot**.

The main backend components are:

```
backend
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com.example.studentmanagement
│   │   │       ├── entity
│   │   │       │   └── Student.java
│   │   │       ├── repository
│   │   │       │   └── StudentRepository.java
│   │   │       ├── controller
│   │   │       │   └── StudentController.java
```

Entity

`Student.java` represents the student table in the database.

Repository

`StudentRepository.java` communicates with the database using Spring Data JPA.

Controller

`StudentController.java` provides REST API endpoints for CRUD operations.

The SOP specifies backend models/entities, REST endpoints, validation, and exception handling as part of backend development.

16. Validation

Validation is used to ensure that users enter appropriate information.

Examples:

- Student name should not be empty.
- Email should not be empty.
- Email should follow a valid email format.
- Department should not be empty.
- Year should be entered.
- Phone number should contain valid information.

Example Java validation:

```
@NotBlank(message = "Name is required")  
private String name;
```

```
@NotBlank(message = "Email is required")  
@Email(message = "Enter a valid email")  
private String email;
```

The SOP requires both appropriate input validation and server-side validation for important fields.

17. Testing

Testing is performed to verify that every component works correctly.

Functional Testing

Test Case	Expected Result
Add student	Student is added
View students	Students are displayed
Search student	Matching records are displayed
Edit student	Student information is updated
Delete student	Student is removed
Empty name	Validation message displayed
Invalid email	Validation message displayed
Invalid ID	Not Found response

18. API Testing Using Postman

Postman is used to test the REST API independently from the frontend.

Example CREATE request

POST `http://localhost:8080/api/students`

Body:

```
{
  "name": "Thiru",
  "email": "thiru@gmail.com",
  "department": "Information Technology",
  "year": 2,
  "phone": "9876543210"
}
```

Expected result

The API should return the newly created student record with its generated ID.

The SOP specifically recommends Postman for testing REST APIs and checking successful responses and error handling.

19. Advantages

- Simple and easy to use.
 - Reduces manual record management.
 - Faster student searching.
 - Easy updating of records.
 - Easy deletion of records.
 - Centralized database storage.
 - REST API-based architecture.
 - Can be extended with additional features.
 - Suitable as a beginner full-stack project.
-

20. Limitations

The basic version has some limitations:

- It does not include advanced authentication.
 - It is designed mainly for basic student record management.
 - Advanced reporting is not included.
 - Role-based access is not included.
 - Cloud deployment may require additional configuration.
-

21. Future Enhancements

The project can be enhanced by adding:

1. Admin login and authentication.
2. Student login.
3. Role-based access control.
4. Student profile pages.
5. Attendance management.
6. Marks and grade management.
7. Course management.
8. Export student records to PDF/Excel.
9. Dashboard with statistics.
10. Cloud database.
11. Deployment to a cloud platform.
12. Email notifications.

The SOP also recommends documenting future enhancements as part of the project documentation.

22. GitHub and Version Control

The project source code can be maintained using Git and GitHub.

Suggested repository structure:

StudentManagementSystem

```
|
|— frontend
|— backend
|— database
|— screenshots
|— README.md
```

The **README.md** should contain:

- Project title
- Project description
- Features
- Technologies used
- Installation steps
- API documentation
- Screenshots
- Future enhancements

The SOP recommends meaningful Git commits, a README, and avoiding passwords/secrets in the repository.

23. Expected Output

After successful execution, the application provides a webpage similar to:

STUDENT MANAGEMENT SYSTEM

```
-----
| Student Name | Email | Department | Year | Phone |
|-----|-----|-----|-----|-----|
```

Name:

Email:
Department:
Year:
Phone:

[SAVE STUDENT]

Search Student:

| ID | Name | Email | Department | Year | Phone | Actions |

| 1 | Thiru | ... | IT | 2 | ... | Edit Delete |

24. Conclusion

The **Student Management System** demonstrates the development of a complete CRUD-based web application using HTML, CSS, JavaScript, Java Spring Boot, REST API, Spring Data JPA, and MySQL.

The application provides essential student management operations such as adding, viewing, updating, deleting, and searching student records. The project also demonstrates frontend-backend communication, database connectivity, validation, API testing, and version control.

Therefore, the project provides practical experience in developing and integrating the major components of a full-stack web application according to the given CRUD application development SOP.